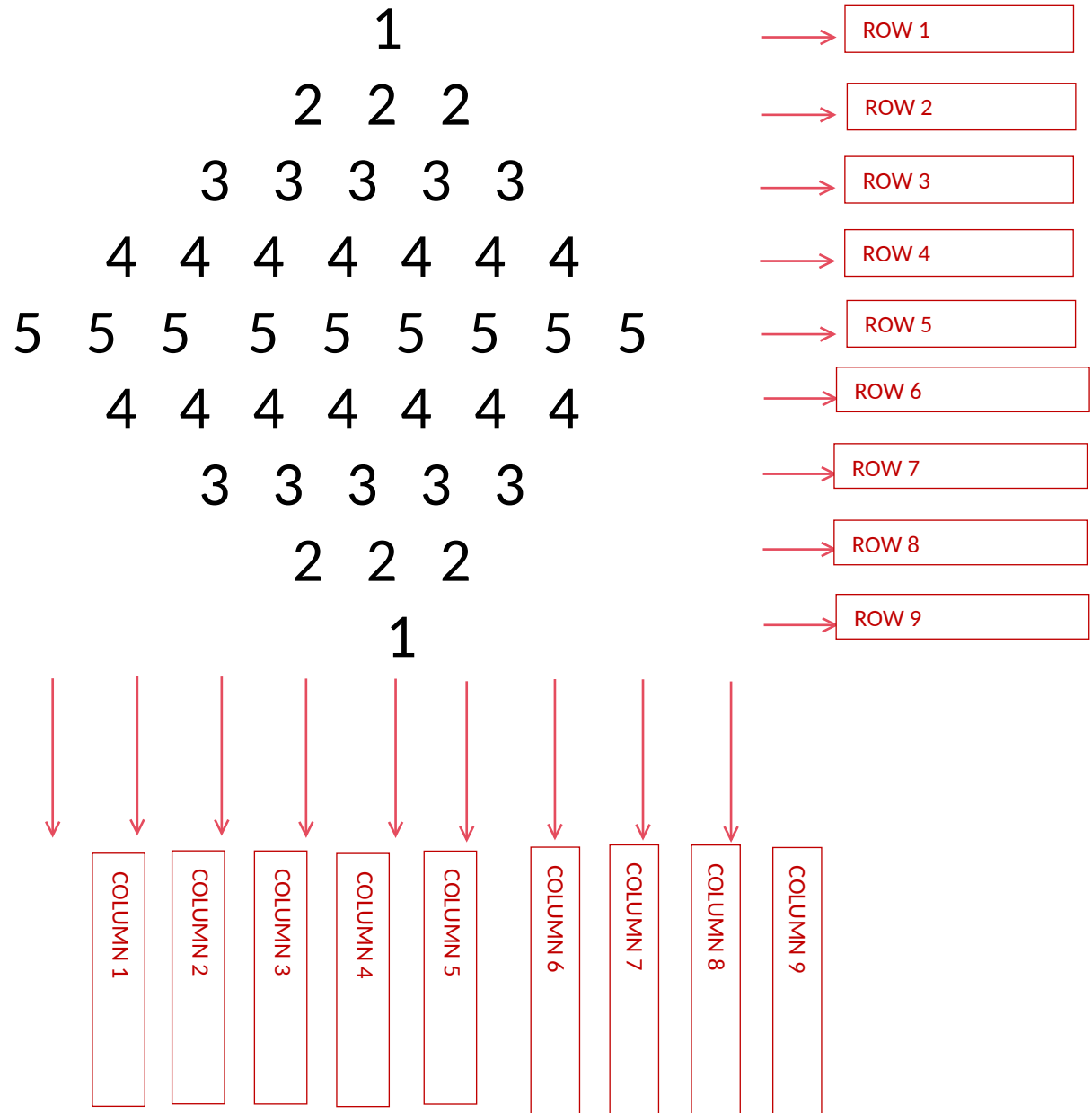


WEEK 01 MODULE
Programming Practice

TOPIC : 2 'Number Pattern'

16. SAME NUMBER DIAMOND = (SAME NUMBER PYRAMID + INVERTED SAME NUMBER PYRAMID)



1. Problem Analysis :-

i. Pattern Structure:

- The output forms a centered same number diamond
- Unlike previous patterns, the diamond is not generated using a single growth rule.

Instead, it consists of two separate parts:

Upper Same Number Pyramid

+

Lower Same Inverted Number Pyramid

- The upper half expands outward from the center.
- The lower half contracts inward toward the center.
- Together, they create a perfectly symmetrical diamond.

ii. Execution Flow:

Step 1: Observe the Grid Concept:

- For $n = 5$

```
      1
     2 2 2
    3 3 3 3 3
   4 4 4 4 4 4 4
  5 5 5 5 5 5 5 5 5
   4 4 4 4 4 4 4
    3 3 3 3 3
     2 2 2
      1
```

Upper Pyramid :-

```
      1
     2 2 2
    3 3 3 3 3
   4 4 4 4 4 4 4
  5 5 5 5 5 5 5 5 5
```

Lower Pyramid :-

```
4 4 4 4 4 4 4
 3 3 3 3 3
  2 2 2
   1
```

- Each cell in the grid represents a unique coordinate (i, j), where i denotes the row index and j denotes the column index.

-The pattern can be viewed as two regions:

✦ Region 1: Leading Spaces

These spaces shift the stars toward the center.

✦ Region 2: Numbers

These form the visible diamond

For n = 5

Row 0 : [][1]

Row 1 : [][2 2 2]

Row 2 : [][3 3 3 3 3]

Row 3 : [][4 4 4 4 4 4 4]

Row 4 : [][5 5 5 5 5 5 5 5 5]

Row 3 : [][4 4 4 4 4 4 4]

Row 2 : [][3 3 3 3 3]

Row 1 : [][2 2 2]

Row 0 : [][1]

UPPER HALF :-

- Spaces decrease by 1.
- Number count increase by 2.

LOWER HALF :-

- Spaces increase by 1.
- Number count decrease by 2.

-> Row Analysis :-

Upper Half :-

Row Index (I)	0	1	2	3	4
Spaces	4	3	2	1	0
number	1	3	5	7	9
Printed value	1	222	33333	4444444	555555555

RELATIONSHIP :-

- Spaces = $n - i - 1$
- Numbers = $2 * i + 1$

Lower Half :-

Row Index (I)	3	2	1	0
Spaces	1	2	3	4
number	7	5	3	1
Printed value	4444444	33333	222	1

RELATIONSHIP :-

- Spaces = $n - i - 1$
- Numbers = $2 * i + 1$

The formulas remain exactly the same for both halves.

Only the direction of row traversal changes.

Upper Half:

$i = 0 \rightarrow n - 1$

Lower Half:

$i = n - 2 \rightarrow 0$

Pyramid (Upper Half):

Spaces ↓

Stars ↑

Inverted Pyramid (Lower Half):

Spaces ↑

Stars ↓

Step 2: Traverse the grid row by row.

- The outer loop is responsible for generating rows.

Generate the upper half using the Pyramid Pattern.

- Traverse rows from 0 to n-1.
- Print the required spaces.
- Print the required numbers.

After the upper pyramid is completed, generate the lower half.

- Traverse rows from n-2 down to 0.
- Print the required spaces.
- Print the required numbers.

Step 3: After all numbers of the current row have been printed, move the cursor to the next line.

Step 4: Repeat the process for all rows until the outer loop terminates.

2. Condition Analysis

No conditional statements are required.

There is no need for if(...)

- Because every position visited by the space loop prints a space.

- Every position visited by the star loop prints a star.

- The pattern is controlled entirely through loop limits.

3. Core Logic Transformation

Pyramid Pattern:-

```
for(int j = 0; j < 2 * i + 1; j++)
```

- **Meaning:**

numbers increase by 2 in every row.

```
for(int j = 1; j <= 2 * (n - i) - 1; j++)
```

Meaning:

Numbers decrease by 2 in every row.

Additionally :-

```
for(int j = 0; j < n - i - 1; j++)
```

- **Meaning:**

controls the alignment of the pattern.

```
for(int i = 0; i < n; i++)
```

Meaning:

Generates the expanding upper pyramid.

```
for(int i = n - 2; i >= 0; i--)
```

Meaning:

Generates the shrinking lower pyramid.

- The formulas for spaces and stars remain exactly the same.
- Only the row traversal direction changes.
- This creates the complete diamond shape.

4. Program :-

```
package week1_module;
import java.util.Scanner;
    public class SameNumberDiamond {

        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter the number of rows: ");
            int n = sc.nextInt();
            // Upper Pyramid
            for(int i = 0; i < n; i++) {

                for(int j = 0; j < n - i - 1; j++) {
                    System.out.print(" ");
                }

                for(int j = 1; j <= 2 * i + 1; j++) {
                    System.out.print((i + 1) + " ");
                }

                System.out.println();
            }

            // Lower Inverted Pyramid
            for(int i = n - 2; i >= 0; i--) {

                for(int j = 0; j < n - i - 1; j++) {
                    System.out.print(" ");
                }

                for(int j = 1; j <= 2 * i + 1; j++) {
                    System.out.print((i + 1) + " ");
                }

                System.out.println();
            }

            sc.close();
        }
    }
```

✦ Why $n - 2$?

Many students write:

```
for(int i = n - 1; i >= 0; i--)
```

and get:

```
      1
    2 2 2
  3 3 3 3 3
4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5
5 5 5 5 5 5 5 5 5
  4 4 4 4 4 4 4
    3 3 3 3 3
      2 2 2
        1
```

here the Middle row is printed twice

as Upper Pyramid already printed:

```
5  5  5  5  5  5  5  5  5
```

So we start from:

$i = n - 2$

which skips the duplicate middle row.

✦ Upper Part Formulas

Spaces: $n - i - 1$

Stars: $2*i + 1$

✦ Lower Part Formulas

Exactly the same formulas.

The only thing changing is:

$i--$ instead of $i++$

which causes the pyramid to shrink.

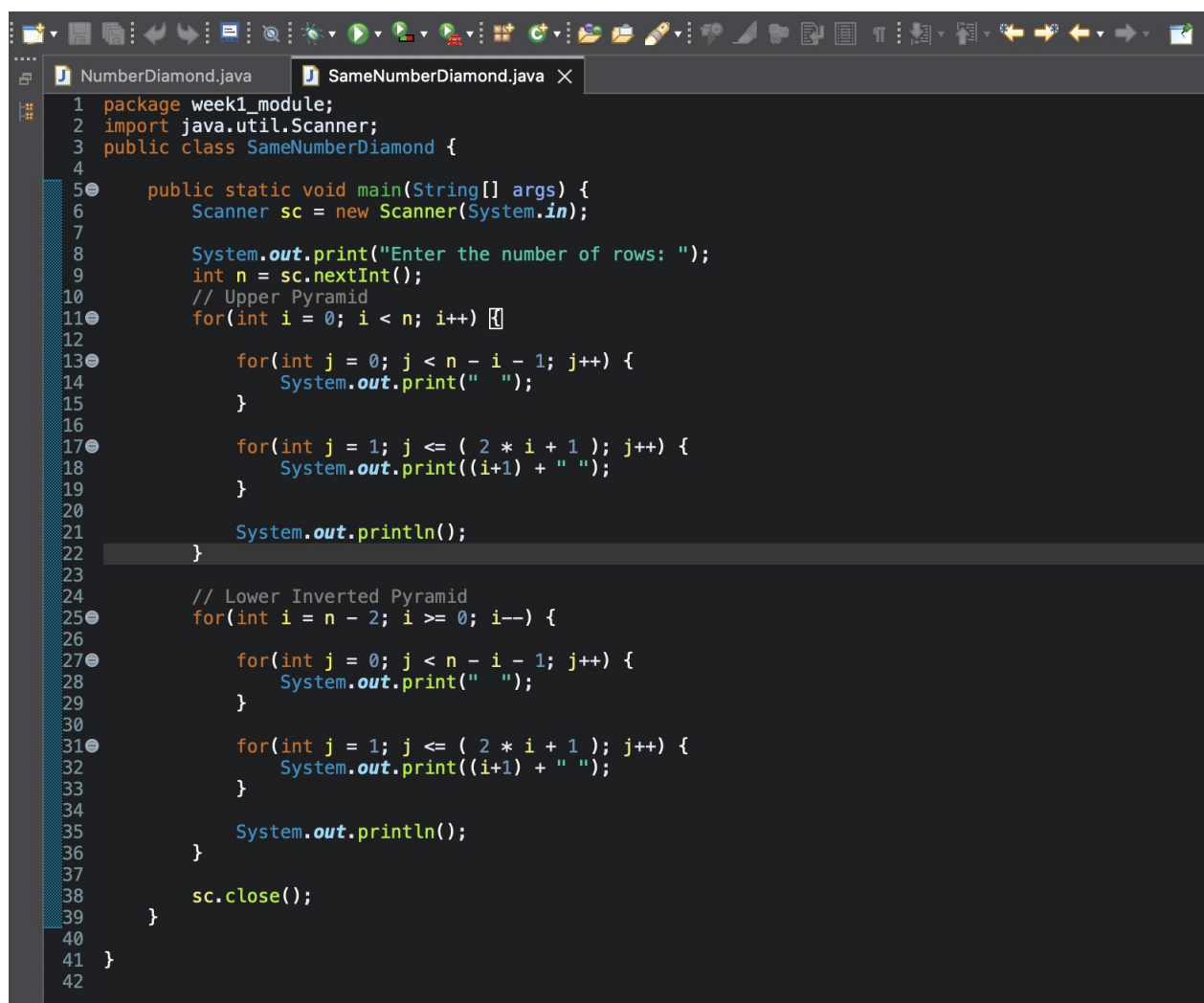
5. Program Output :-

Enter the number of rows: 5

```
    1
  2 2 2
3 3 3 3 3
4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5
4 4 4 4 4 4 4
3 3 3 3 3
2 2 2
1
```

6. ScreenShot :-

i. Program :-



```
1 package week1_module;
2 import java.util.Scanner;
3 public class SameNumberDiamond {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         System.out.print("Enter the number of rows: ");
9         int n = sc.nextInt();
10        // Upper Pyramid
11        for(int i = 0; i < n; i++) {
12
13            for(int j = 0; j < n - i - 1; j++) {
14                System.out.print(" ");
15            }
16
17            for(int j = 1; j <= ( 2 * i + 1 ); j++) {
18                System.out.print((i+1) + " ");
19            }
20
21            System.out.println();
22        }
23
24        // Lower Inverted Pyramid
25        for(int i = n - 2; i >= 0; i--) {
26
27            for(int j = 0; j < n - i - 1; j++) {
28                System.out.print(" ");
29            }
30
31            for(int j = 1; j <= ( 2 * i + 1 ); j++) {
32                System.out.print((i+1) + " ");
33            }
34
35            System.out.println();
36        }
37
38        sc.close();
39    }
40
41 }
42
```

ii. Program's Output :-

```
Problems @ Javadoc Declaration Console X
<terminated> SameNumberDiamond [Java Application] /Library/Java/JavaVirtualMachines/temurin
Enter the number of rows: 5
      1
     2 2 2
    3 3 3 3 3
   4 4 4 4 4 4 4
  5 5 5 5 5 5 5 5 5
   4 4 4 4 4 4 4
    3 3 3 3 3
     2 2 2
      1
```